

CO4 AT1 CASE STUDY

Name : Vedvyass M

Reg no : 192324267

Case Study 5: University Database Schema Evaluation

1. Analysis of the Existing Schema and Design Problems

The university currently maintains information using four main tables: **Student, Course, Faculty, and Department**. The existing design has several problems related to redundancy, consistency, and maintainability.

Major design problems

- 1. Faculty information redundancy**
Faculty details such as name, email, phone number, and department may be repeated in several course records. If a faculty member teaches multiple courses, the same information is stored repeatedly.
 - 2. Department duplication**
Department information is duplicated across student and faculty records. This increases storage requirements and creates opportunities for inconsistent department names or details.
 - 3. Update anomaly**
When a faculty member changes their phone number or email address, the information must be updated in multiple records. Missing even one update can result in conflicting information.
 - 4. Insertion anomaly**
It may be difficult to add a new faculty member or department if the current design requires an associated course or student record.
 - 5. Deletion anomaly**
Removing the last course associated with a faculty member could unintentionally remove important faculty information.
 - 6. Conflicting reports**
Because the same information is stored in multiple places, different reports may produce different values for the same faculty or department.
-

2. Redundancy and Data Integrity Issues

The major redundancy occurs because information describing the same entity is stored repeatedly.

For example, consider a faculty member teaching three courses:

Course	Faculty Name	Faculty Email	Faculty Phone
Database Systems	Dr. Kumar	kumar@univ.edu	9876543210
Data Mining	Dr. Kumar	kumar@univ.edu	9876543210
AI	Dr. Kumar	kumar@univ.edu	9876543210

The faculty information is unnecessarily repeated three times.

If the phone number changes and only two records are updated, the database could contain:

Database Systems → 9876543210
Data Mining → 9876543210
AI → 9123456780

This creates a **data integrity problem**.

Types of anomalies

Update anomaly:

The same information must be changed in several records.

Insertion anomaly:

New entities cannot easily be inserted without unrelated information.

Deletion anomaly:

Deleting one record may accidentally remove information about another entity.

The solution is to store each entity's information in an appropriate table and connect the tables using primary and foreign keys.

3. Explanation of Relationships Among Entities

The major entities and their relationships can be represented as follows:

Department–Student

One department can have many students.

Department 1 ————— N Student

For example, the Computer Science department can contain hundreds of students.

Department–Faculty

One department can have many faculty members.

Department 1 ————— N Faculty

Department–Course

One department can offer multiple courses.

Department 1 ————— N Course

Faculty–Course

A faculty member may teach multiple courses, and a course may potentially be taught by multiple faculty members depending on the university structure.

For a many-to-many relationship, an intermediate table should be introduced:

Faculty 1 — N Faculty_Course N — 1 Course

Student–Course

A student can enroll in multiple courses, and a course can contain multiple students.

Therefore:

Student N — M Course

This many-to-many relationship should be implemented using an intermediate table such as Enrollment.

4. Recommended Improved Schema

A normalized database design can contain the following tables.

Department

Department

Department_ID (PK)

Department_Name

Office_Location

Student

Student

Student_ID (PK)

Student_Name

Email
Department_ID (FK)

Faculty

Faculty

Faculty_ID (PK)

Faculty_Name

Email

Phone

Department_ID (FK)

Course

Course

Course_ID (PK)

Course_Name

Credits

Department_ID (FK)

Enrollment

Enrollment

Student_ID (PK, FK)

Course_ID (PK, FK)

Semester

Grade

Faculty_Course

Faculty_Course

Faculty_ID (PK, FK)

Course_ID (PK, FK)

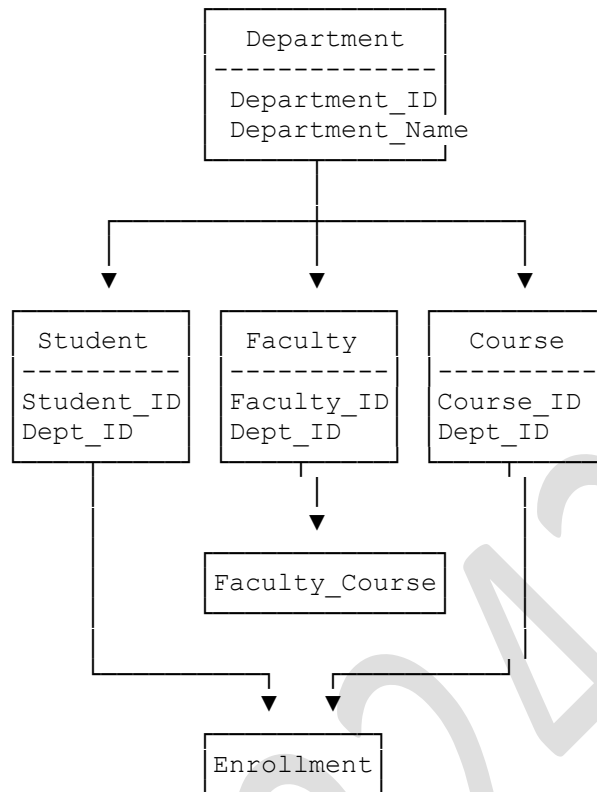
Semester

OUTPUT :

Department_Name	Student_Count
Computer Science	250
Information Technology	180
Artificial Intelligence	220
Electronics	170
Mechanical	180

5. Improved Database Relationship

The redesigned structure can be represented as:



This design separates information according to its entity and uses relationships instead of repeating the same data.

6. Normalization Strategy

The redesigned schema should follow normalization principles, particularly **First Normal Form (1NF)**, **Second Normal Form (2NF)**, and **Third Normal Form (3NF)**.

First Normal Form

Each column should contain atomic values, and repeating groups should be removed.

Second Normal Form

Non-key attributes should depend on the complete primary key.

This is particularly important for tables such as **Enrollment**, where the combination of **Student_ID** and **Course_ID** can form a composite key.

Third Normal Form

Non-key attributes should depend only on the primary key and not on another non-key attribute.

For example, department details should be stored in the `Department` table rather than repeatedly in `Student`, `Faculty`, and `Course` tables.

7. Benefits of the Redesigned Schema

The proposed schema provides several advantages:

- Reduces duplicate data.
- Prevents inconsistent faculty information.
- Makes updates easier.
- Improves data integrity.
- Reduces storage requirements.
- Clearly represents relationships between entities.
- Prevents update, insertion, and deletion anomalies.
- Makes future expansion easier.
- Supports reliable reporting and analytics.
- Provides better maintainability.

For example, if a faculty member changes their phone number, the administrator only needs to update the `Faculty` table once.

8. Support for Analytics and Decision-Making

A well-designed database improves the reliability of university analytics.

The university can generate reports such as:

- Number of students in each department.
- Faculty workload.
- Number of courses offered by each department.
- Student enrollment by course.
- Student performance by course.
- Faculty teaching assignments.
- Department-wise academic performance.
- Course popularity and enrollment trends.

Because redundant information is removed, analytical queries are less likely to produce conflicting results.

For example, the university can calculate department-wise student enrollment using:

```
SELECT
    d.Department_Name,
    COUNT(s.Student_ID) AS Student_Count
FROM Department d
LEFT JOIN Student s
    ON d.Department_ID = s.Department_ID
GROUP BY d.Department_Name;
```

Similarly, faculty workload can be analyzed using the `Faculty_Course` relationship.

9. Final Recommendation

The existing university database should be redesigned using a **normalized relational schema**. Faculty, department, student, and course information should be stored independently, while relationships should be represented using foreign keys and associative tables such as `Enrollment` and `Faculty_Course`.

The most important improvement is to eliminate repeated faculty and department information. This reduces redundancy and prevents update, insertion, and deletion anomalies.

The redesigned schema will provide **better data integrity, consistency, scalability, and analytical capability**. It will also allow the university to generate reliable reports and support future decision-making without maintaining the same information in multiple locations.

Conclusion

The existing schema has significant redundancy and integrity problems because faculty and department information is repeated across multiple records. A normalized schema with appropriate primary keys, foreign keys, and relationship tables provides a more efficient solution. The proposed design separates entities clearly and represents their relationships accurately. As a result, the database becomes easier to maintain, less prone to inconsistencies, and more suitable for future university analytics and decision-making.